ABES Institute of Technology
Ghaziabad

Python Programming

Unit-1 : Introduction

Lecture -1

Content: What is Python programming language and its programming cycle, Python IDE.

Lecture Outcome: Students will be able to know about python programming, its history and some of Python IDE and programming cycle of python.

(1) 1.1) What is Python:

•) Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Python was developed by Guido Van Rossum in the late eighties and early nineties (1991) at the National Research Institute for Mathematics and Computer Science in the Netherlands.

•) Python programming language (latest Python 3) is being used in web development, Machine learning applications, along with all cutting edge technology in Software industry.

There are two major Python versions: Python 2 and Python 3

1.2) Features of Python:-

- Less line of codes than other languages (fast execution)
- Platform independent

- Open Source
- Object Oriented
- It supports functional and structured programming methods

It can be easily integrated with C, C++, COM, CORBA and Java

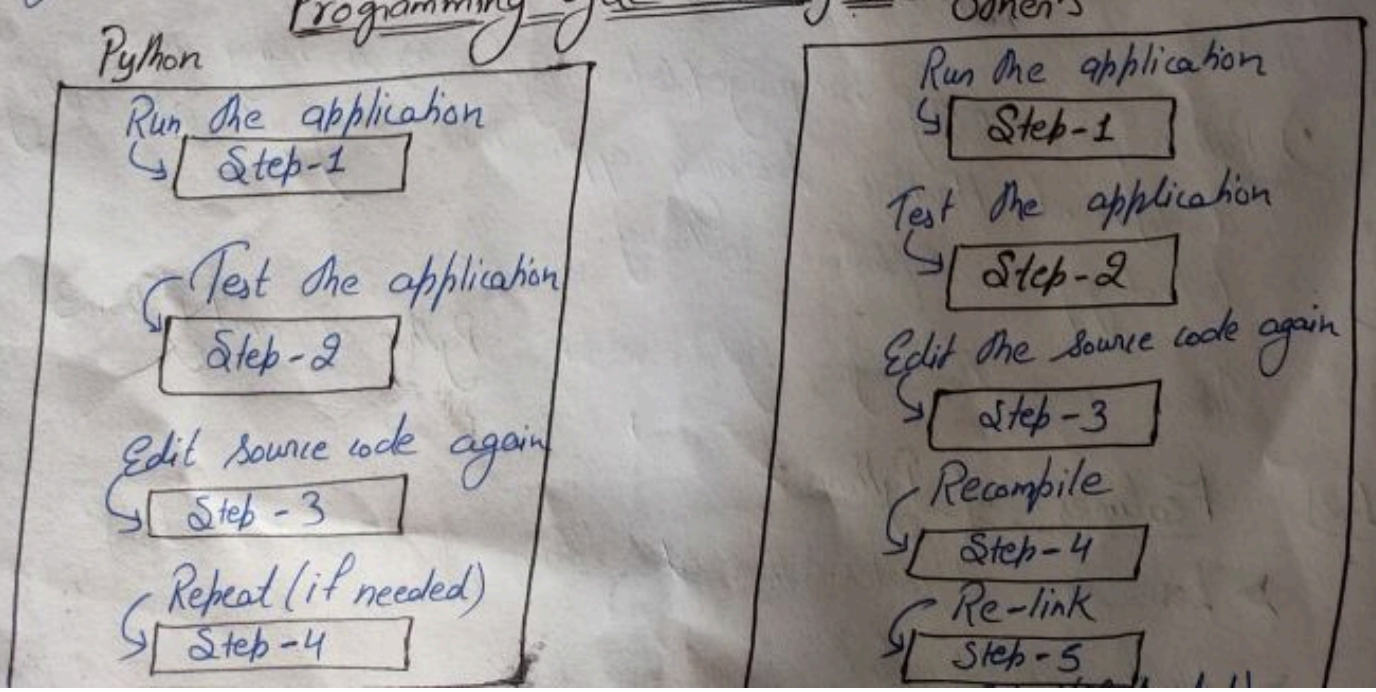
Organizations using Python:

1. Google (Components of Google Spider and Search Engine)
2. Yahoo (Maps)
3. Youtube
4. Mozilla
5. Dropbox
6. Microsoft
7. Cisco
8. Spotify
9. Quora

② What is the Programming Cycle for Python?

The development cycle of Python is considerably shorter than that of traditional tools. There are no compilation or linking steps in Python. Python programs simply import modules at runtime and use the objects they contain. Because of this, Python programs run immediately after changes are made.

Programming Cycle for Python



(3) Python IDE

Python code editors are designed for the developers to code and debug program easily. Using these Python IDEs (Integrated development Environment), we can manage a large codebase and achieve quick deployment.

1) **PyCharm:**

Pycharm is a cross-platform IDE used for Python programming. It is one of the best Python IDE Editor that can be used on Windows, macOS and Linux. This software contains API that can be used by the developers to write their own Python plugins so that they can extend the basic functionalities.

Price: Free/Paid

2) **Kite:**

Kite is IDE for Python that automatically completes multiple line codes. This editor support more than 16 languages. It helps you to code faster with no hassle.

Price: Free

3) **Spyder:**

Spyder is a scientific integrated development environment written in Python. This software is designed for and by scientists who can integrate with Matplotlib, SciPy, NumPy, Pandas, Cython, IPython, SymPy and other open-source software. Spyder is available through Anaconda (open-source distribution System) distribution on Windows, macOS and Linux.

Price: Free

) **IDLE:**

IDLE (Integrated Development and learning Environment) is a default editor that comes with Python. It is one of the best Python IDE software which helps a beginner to learn Python easily.

IDLE software package is optional for many Linux distributions. The tool can be used on Windows, macOS and Linux.

Price: free

5) Sublime Text 3

Sublime Text 3 is a code editor which supports many language including Python. It is one of the best Python editor that has basic built-in support for Python.

Price: Free Trial

6) Visual Studio Code

Visual Studio Code (VS Code) is an Open-Source environment developed by Microsoft. It is one of the best Python IDE for Windows that can be used for Python development.

Price: Free

7) Atom

Atom is a useful code editor tool preferred by programmers due to its simple interface compared to the other editors.

Price: free

8) Jupyter

Jupyter is tool for people who have just started with data science. It is easy to use, interactive data science IDE across many programming language that just work as an editor, but also as an educational tool or presentation.

Price: Free

9) PyDev

PyDev is a third-party python editor for Eclipse. It is one of the best IDE which can be used in not only Python but IronPython

ABES Institute of Technology
Ghaziabad

Python Programming

Unit-1 : Introduction Lecture-2

Content: Interacting with Python PROGRAMS, Elements of Python

Lecture Outcome: Students will able to know how to install Python and interact with basic program of Python.

→ Installing Python

Step-1 : Open a browser and open https://www.python.org
When you click on the download button, python can be installe right after you complete the setup by executing the file for your platform.

Step-2 Download Visualstudio Code IDE

Open a browser and open https://code visualstudio.com
When you click on the download button, python can be installed right after you complete the setup by executing the file for your platform.

⇒ Interacting with first Python Programs

- Python programs must be written with a particular structure. The syntax must be correct, or the interpreter will generate error messages and not execute the program. This section introduces Python by providing a simple example program.

Example 1 Hello.py

```
print(" Hello class, hope you all are doing great")
```

Python Elements

⇒ 1. Python Identifiers

- A python identifiers is a name used to identify a variable, function, class, module or other object.
- An identifier starts with letter A to Z or a to z or an underscore (-) followed by zero or more letters, underscores and digits (0 to 9).
- Python does not allow punctuation characters such as @, $ and % within identifiers. Python is a case sensitive programming language.

⇒ 2. Reserved Words

Reserved words can't use as constant or variable or any other identifiers names. All the python keywords contain lowercase letters only, so list of Python Keywords as mention on next page.

and	exec	not	assert	finally	or
break	for	pass	class	from	print
continue	global	raise	def	if	return
del	import	try	elif	in	while
else	is	with	except	lambda	yield

⇒3. Lines and Indentation

Python provides no braces to indicate block of code for class and function definitions or flow control. Block of code are denoted by line indentation, which is rigidly enforced. The number of spaces in the indentation is variable, but all Statements within the block must be indented the same amount.

For example -

```
if True:
    print("Welcome")

else:
    print("Bye")
```

However, the following block generates an error

```
if True:
print("Welcome")
else:
print("Bye")
```

⇒4 Multi- Line Statements

Statement in Python typically end with a new line. However, Python allow the use of the line continuation character (\) to denote that the line should continue. For example:-

```
Sum = item-1 + \
      item-2 + \
      item-3
```

Statement contained within the [], {} or () brackets do not need the line continuation character. For example.

```
Fruits = ['Mango', 'Banana',
          'Guava']
```

⇒5. Quotation in Python

- Python accepts single(' '), double(" ") or triple(''' or """) quotes to denote string literals, same type of quotes starts and ends the string.
- The triple quotes are used to span the string across multiple lines. Ex-

```
paragraph= """ From here paragraph has started. It is
               the collection of multiple lines and conclusion
               of story."""
```

⇒6. Comments in Python

A has sign (#) that is not inside a string literal begins a comment. All character after the # and up to the end of the ~~py~~ physical line are part of the comment.

Example

```
#First Comment
print("Hello, Universe")
```

- We can type a comment on the same line after a statement.

```
print("Hello, India") # Second comment.
```

- Triple-quoted can be used as a multiline comments.

⇒ 7. Variables

- Variables are reserved memory locations to store values. When we create a variable we reserve some space in memory.
- By assigning different data types to variables, we can store integers, decimals or characters in these variables.

8) Assigning Values to Variable

- Python variables do not need explicit declaration to reserve memory space. The declaration happens automatically when we assign a value to a variable.
- (=) equal sign is used to assign values to variables.

Example :

```
number = 25      # An integer assignment.
circle = 3.14    # floating point number
name = "Abesit"  # String
```

⇒ 9. Standard Data types

Python has five standard data types -

1) Number
   - int
   - long
   - float
   - complex
2) String ⇓ Set of characters
3) List
4) Tuples
5) Dictionary

⇒ 7.1 Python lists

•) A list contain item seperated by commas and enclosed within square brackets [].

•) To some extent, lists are similar to arrays in C. One difference between them is that all the items belonging to a list can be of different data type.

Example:

list = ['Ram', '486', '45.352', 'xyz']

⇒ Python Tuples

•) Tuples is similar to list, the only differences are, lists are enclosed in brackets [] and their element and size can be changed, while tuples are enclosed in parantheses () and can't be updated.

tuple = ('Ram', '486', '45.352', 'xyz')

⇒ Type Conversion

Python defines type conversion functions to directly convert one data type to another which is useful.

There are two types of Type Conversion in Python:

1) Implicit Type Conversion 2) Explicit Type Conversion

1) Implicit Type Conversion:

•) In Implicit Type Conversion of data types, the ~~Phy~~ Python interpreter automatically converts one data type to another without any user involvement.

Example:

```
x = 50
y = 70.5
Sum = x+y
print(sum)
print(type(sum))
print("x is of type:", type(x))
print("y is of type:", type(y))
```

Output

```
120.5
<class 'float'>
x is of type: <class 'int'>
y is of type: <class 'float'>
```

2) Explicit Type Conversion

- In Explicit Type conversion, the data type is manually changed by the users as per the requirement.
- This conversion form is also called typecasting as the user cast (changes) the object data types.

1. int(a, base): This function converts any data type to integer. 'Base' specifies the base in which string is if the data type is a string.
2. float(): This function is used to convert any data type to a floating-point number.

Example

```
x = 50
y = "700"
result1 = x+y
print(result1)
y ~~result2~~ = int(y)
result2 = x+y
print(result2)
```

Output

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

750

3. ord(): This function is used to convert a character to integer.
4. hex(): This function is to convert integer to hexadecimal string.
5. Oct(): This function is to convert integer to Octal string.
6. tuple(): This function is used to convert to a tuple.
7. Set(): This function returns the type after converting to set.
8. list(): This function is used to convert any data type to a list type.
9. dict(): to convert a tuple of order (Key, value) into a dictionary.
10. Str(): Used to convert integer into a string.
11. Complex(real, imag): Convert real numbers to complex (real, imag) number.
12. chr(number): Converts number to its corresponding ASCII character.

<u>Example</u>:

```
a = chr(76)
b = chr(77)
print(a)
print(b)
```

O/P

```
L
M
```

ABES Institute of Technology
Ghaziabad
Python Programming

Lecture : 3

Unit-1 : Basics

Content: Expressions, Assignment Statement, Arithmetic Operators, Operators Precedence

Lecture Outcome: Students will able to Know expressions, assignment Statement, arithmetic operators and operators precedence of python.

•) **Expression**: An expression is a combination of values, variables, and operators in our program that evaluates to some value.

Example : 2+5
7

•) **Assigment Statements in Python**

Python assignment statements to assign objects to names.

There are some properties of assignment in python :→

•) Assignment creates object references instead of copying the objects.

•) Python creates object a variables name the first time when they are assigned a value.

•) Names must be assigned before being referenced.

- There are some operations that perform assignment explicitly.

Assignment Statements forms:-

1. Basic form: Most common form

Example- class = 'Second year'
print ('class')

O/P class

2. Tuple Assignment:

Ex: a, b = 2, 4
print ('a = ', a)
print ('b = ', b)

O/P
a = 2
b = 4

3. When we code a tuple on the left side of the = Python pairs objects on the right side with target on the left by position and assign them from left to right. Therefore the values of a and b are 2 and 4 respectively.

3. List Assignment: Same as tuple assignment.

Ex: [x, y] = [2, 4]
print ('x = ', x)
print ('y = ', y)

O/P
x = 2
y = 4

4. Sequence Assignment

Ex: a, b, c = 'WOW'
print ('a = ', a)
print ('b = ', b)
print ('c = ', c)

O/P
a = W
b = O
c = W

5. Extended Sequence unpacking:

Ex: p, *q = 'Hello'

print ('p =', p)

print ('q=', q)

O/P

p = H

q = ['e', 'l', 'l', 'o']

Here, p is matched with the first character in the string on the right and q with the rest. The starred name (*q) is assigned a list, which collects all items in the sequence not assigned to other names.

6. Multiple-target assignment:

Ex: a = b = 4

print(a, b)

O/P 4, 4

Python assigns a reference to the same object to all the target on the left.

7. Augmented assignment:

Ex: x = 2

x += 1 # equal to x = x + 1

print(x)

O/P = 3

The augmented assignment combines an expression and an assignment

⇒ Operators

Operators are the constructs which can manipulate the value of operands.

Consider the expression 3 + 7 = 10, where 3 and 7 are called operands and + is called operators.

Types of Operator

1) Arithmetic Operator

2) Comparison (Relational) Operators.

3) Assignment Operators
4) Logical operators
5) Bitwise Operators
6) Membership Operators
7) Identity operators

Python Arithmetic Operators

Let a variable holds 10 and variable b holds 20, then

Operators	Description	Example
+ Addition	Add values on either side of the operator	a+b=30
- Subtraction	Subtracts right hand operand from left hand operand.	a-b = -10
* Multiplication	Multiplies values on either side of the operator	a*b=200
/ Division	Divides left hand operand by right hand operand	b/a = 2
% Modulus	Divides left hand operand by right hand operand and returns remainder.	b%a =0
** Exponent	Perform exponential (power) Calculation on operators	a**b =10 to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digit after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e, rounded away from zero.	9/2 =4 and 9.0/2.0 = 4.0, -11/3 = -4, -11.0/3 = -4.0

Python Comparison Operators

Let variable a holds 10 and variable b holds 20, then

Operator	Description	Example
==	If the value of two operands are equal, then the condition become true.	(a==b) is not true
!=	If values of two operand are not equal, then condition becomes true	(a!=b) is true
<>	If values of two operands are not equal, then condition becomes true	(a<>b) is true. It is similar to != operator
>	If the value of left operand is greater than the value of right operand, then condition is true	(a>b) is not true.
<	If the value of left operand is less than the value of right operand, then condition is true	(a<b) is true
>=	If the value of operand is greater than or equal to the value of right operand, then condition becomes true.	(a>=b) is not true
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true	(a<=b) is true

Python Assignment Operators

Variable a holds 10 and variable b holds 20, then -

Operator	Description	Example
=	Assign values from right side operands to left side operand.	c=a+b assign value of a+b into c c=30
+= Add AND	It adds right operand to the left operand and assign the result to left operand.	c+=a is equal to c=c+a c=40

-= Subtract AND	It subtract right operand from the left operand and assign the result to the left operand.	c-=a is equal to c=c-a
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand.	c*=a is equal to c=c*a
/= Divide AND	It divides left operand with the right operand and assign the result to left operand	c/=a is equal to c=c/a
%= Modulus AND	It takes modulus using two operands and assign the result to left operand	c%=a is equal to c=c%a
**= Exponent AND	Perform exponential(power) calculation on operators and assign value to the left operand.	c**=a is equal to c=c**a
//= Floor Division	It perform floor division on operators and assign value to the left operand	c//=a is equal to c=c//a

6) Python Bitwise Operators

a=2, b=3
a=0010, 0011

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) means a = 0010, b = 0011 o/p (0010)
| Binary OR	It copies a bit if it exists in either operand.	(a|b) a=0010, b=0011, o/p (0011)
~ Binary Ones (ones) Complement	It is unary and has the effect of 'flipping' bits	a = 0010, ~a = 1101
<< Binary left shift	The left operand value is moved left ~~right~~ by the number of bits specified by the right operand.	a<<2, a = 0010, a<2 = 1000

>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a = 0010 a>>2 = 0000
^ Binary XOR	It copies the bit if it is set in one operand but not both.	a = 0010 b = 0011 a^b = 0001

Python Logical Operators

Operator	Description	Example
and Logical AND	If both operands are true then condition become true.	(a and b) is true
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true
not Logical NOT	Used to reserve the logical state of its operand.	Not (a and b) is false

Python Membership Operators

Operators	Description	Example
~~Not~~ in	Evaluate to true if it ~~does not~~ finds a variable in the specified sequence and false otherwise.	x in y, here in a 1 if x is a member of sequence y.
not in	Evaluate to true if it does not finds a variable in the specified sequence and false otherwise	x not in y, here not in result in a 1 if x is not a member of sequence y

Python Identify Operators

Operator	Description	Example
is	Evaluate to true if the variables on either side of the operator point to the same object and false otherwise	x is y, here is results in 1 if id(x) equal id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).

Python Operators Precedence

S.No	Operator & Description
1	** Exponentiation (raise the power)
2	~+- Complement, unary plus and minus (method names for the last two are +@ and -@)
3	*/%// Multiply, divide, modulo and floor division
4	+- Addition and subtraction
5	>> << Right and left bitwise shift
6	& Bitwise 'AND'
7	^| Bitwise exclusive 'OR' and regular 'OR'
8	<=, <>, >= Comparison operators
9	<>, ==, != Equality operators
10	=, %=, /=, //=, -=, +=, *=, **= Assignment operators
11	is, is not Identity operators
12	in, not in Membership operators.
13	not, or, and Logical operators.

ABES Institute of Technology
Ghaziabad

Python Programming

Unit 1: Basics Lecture - 4

Content: Boolean Expression

Lecture Outcome: Students will able to know boolean expression and learn some basic program also write programs in python.

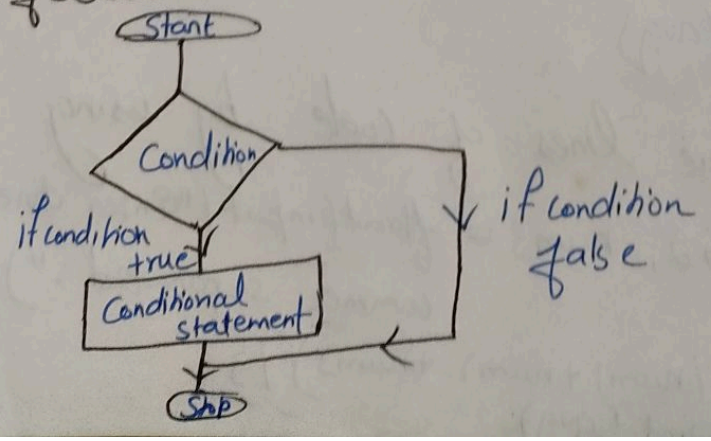
Boolean Values

- In Python, the two Boolean values are True and False
- When we evaluate any expression in Python, get one of two answers i.e 'True' or False

Example:

```
a = 4
b = 3
if b > a :
    print(" b is greater than a")
else :
    print (" b is not greater than a")
```

Structure of Boolean Expression



•) Python Programming language assumes any non-zero and non-null values as True, and if it is either zero or null, then it assumed as False value.

Ex-1

Q To make simple calculator, code written in python.

```
a = float(input("Enter your first number : "))
b = float(input("Enter your second number : "))
print (" Summation=", (a+b))
print ("Subtraction =", (a-b)
print (" Multiplication =", (a*b))
print (" Division=", (a/b))
print (" floor division = ", (a//b))
```

Q Ex-2 Finding average of three number.

```
num1 = float(input("Enter your first number:"))
num2 = float(input("Enter your second no. :"))
num3 = float(input(" Enter your third no. "))
avg = (num1 + num2 + num3)/3
print(avg)
```

Trick to reduce lines of code by using split function.

```
num1, num2, num3 = float(input("Enter three number using
                    comma separated :").split(",")
avg = (num1+num2+num3)/3
print (avg)
```

Ex-3 Check condition by using if statement

```
age = int(input("Enter your age : "))
if age >= 20
    print("you are eligible to fill the form.")
```

Ex-4 Python IDE use a simple calculator.

```
print(2*3*4+5)
print(2**3**4)
```